

Site déjà largement en place. Insister sur les corrections, pas tout redétailler.

POINTS CORRIGÉS À MENTIONNER

C1.B — RESPONSIVE (ÉTAIT 0/3)

Cause racine trouvée : `body{display:flex}` empêchait `<main>` de rétrécir → un seul élément large faisait déborder toute la page. Corrigé (`flex:1 1 auto; min-width:0`), puis retravaillé : formulaire recherche, nav mobile (9 liens → 3 + menu), tableaux RGPD scrollables, images allégées <540px. Vérifié à 1920/768/375px.

C2.D — LIBRAIRIE JS EXTERNE (ÉTAIT 0/3)

Swiper.js pour le carrousel (au lieu de recoder swipe tactile + clavier + pagination moi-même). Hébergé en local, pas en CDN, pour éviter une panne le jour J.

C1.C — ACCESSIBILITÉ

Police OpenDyslexic activable (mémorisée), curseur "main" retiré des cartes non cliquables.

C2.C — REQUÊTES ASYNCHRONES RENFORCÉES

Vrais verbes HTTP variés en AJAX : PUT (profil), PATCH (mot de passe — modif partielle vs PUT qui remplace tout), DELETE (suppression compte), avec jeton CSRF en en-tête.

Si le jury creuse

Bug de balisage trouvé sur 16 pages : `<button><a>...</button></a>` (mal fermé, casse le clavier) → remplacé par un simple `<a>`. Aussi : `vol.nom` n'existait pas en base (colonne réelle : `vol.ville` via jointure destinations) → titres de cartes vides corrigés.

BLOC_1_Front-End.pdf pour le détail complet

Seul manque signalé par le jury : pas d'espace admin avec vrai CRUD. C'est LE point à traiter en premier.

LE CORRECTIF PRINCIPAL

C5.C — ESPACE D'ADMINISTRATION COMPLET (AJOUTÉ)

4 routes : `GET /admin/comptes` (lister), `POST` (ajouter), `PUT /admin/api/comptes/<id>` (modifier), `DELETE` (supprimer). Mise en page séparée (sidebar dédiée). Protégé par `admin_required`, vérifié avec de vraies requêtes HTTP sur 2 comptes (visiteur → login, client non-admin → 403).

POINTS FORTS DÉJÀ ACQUIS (2/3 CHACUN)

C5.A — DOC DU FRAMEWORK

3 extensions officielles : Flask-Login, Flask-WTF (CSRF), Flask-Limiter. Décorateur maison
`admin_required = @login_required + vérif is_admin`.

C5.B — DÉPENDANCES

`requirements.txt` versions figées. Config sensible centralisée dans `.env` (14 variables), jamais commité.

Si le jury creuse (bug de sécurité réel, à bien savoir expliquer)

L'ordre des colonnes SQL ne correspondait plus au code Python → risque qu'un simple client apparaisse comme admin. Corrigé en passant à `RealDictCursor` (accès par nom de colonne `user['email']`, jamais par position) pour éliminer ce risque définitivement.

Aussi : 25 `try/except` + 23 `finally` dans `app.py` → connexion DB toujours fermée, jamais de trace technique montrée au visiteur.

BLOC_3_Framework.pdf pour le détail complet

Reproché : framework utilisé au lieu de code natif + aucun rôle utilisateur + pas de CRUD complet. Tout a été réécrit en Python pur.

L'ESSENTIEL À DIRE D'EMBLÉE

LE POINT CENTRAL

Backend entièrement réécrit en **Python natif** (module `http.server` de la bibliothèque standard) — **aucun Flask, aucun framework**. Routage, sessions, formulaires : tout fait main.

CE QUI A ÉTÉ CONSTRUIT (PAR COMPÉTENCE)

C3 — DONNÉES

Modèle de données documenté (schéma + relations), base PostgreSQL construite, requêtes SQL paramétrées (lister/ajouter/modifier/supprimer + jointures + tri/filtres), RGPD (voir ci-dessous).

C4.C/D — P00 + MVC

`ModeleBase` → `CompteClient` (héritage réel).
3 dossiers stricts : `models/` (BDD), `views/` (HTML pur), `app.py` (contrôleur).

C4.E — RÔLES

`is_admin` détermine le rôle. Session = jeton aléatoire (`secrets.token_hex(32)`) stocké serveur, expire à 30 min. Cookie `HttpOnly` — jamais les vraies données côté client.

LE CRUD COMPLET (POINT REPROCHÉ)

Espace admin :
lister/ajouter/**modifier/supprimer** n'importe quel compte. Anti-auto-lockout : un admin ne peut ni se retirer ses droits ni se supprimer lui-même.

RENFORCEMENTS AU-DELÀ DU STRICT MINIMUM

RGPD SELF-SERVICE (C3.D)

Un client voit/modifie/supprime **lui-même** son compte (mot de passe requis pour supprimer), sans dépendre d'un admin. Isolation vérifiée : impossible de cibler un autre compte même en injectant un id dans le formulaire (l'id vient toujours de la session, jamais du client).

SÉPARATION STRICTE DES ESPACES

Testé dans les deux sens : client → `/admin` = 403 ; admin → `/client` = redirigé vers son propre espace.

SÉCURITÉ

Rate-limiting maison (5 tentatives/min sur login → 429), échappement HTML systématique (anti-XSS), requêtes 100% paramétrées (anti-injection SQL).

API EXTERNE (CR 3.A.3)

Open-Meteo (géocodage + météo), appelée en `urllib` natif. Affichée en direct sur le dashboard client.

TESTS (C4.G)

10 tests unitaires (`unittest` , natif) sur l'échappement, le rate-limit, les sessions — en plus des tests fonctionnels bout-en-bout faits pendant le dev.

Repères pour naviguer vite dans le code si le jury demande une modif en direct

`app.py` = routes + logique (contrôleur) · `models/compte_client.py` = toutes les requêtes SQL sur les comptes · `views/admin.py` / `views/client.py` = HTML affiché · `Database/db.py` = connexion PostgreSQL.

brouillon_bloc2.md pour le détail complet par compétence